Facultad de Informática

UNIVERSIDAD DE MURCIA

PRÁCTICA DE AVANCE RÁPIDO Y BACKTRACKING

Profesor: JOAQUÍN CERVERA LÓPEZ

Nombre	Grupo	Subgrupo	e-mail	DNIs
María Almarcha Lorca	2	2.2	m.almarchalorca@um.es	34332923H
Fadwa Abbaq El Gharra	2	2.2	fadwa.a.g@um.es	30850969B

ÍNDICE

LISTA DE PROBLEMAS RESUELTOS **2**
AVANCE RÁPIDO: RESOLUCIÓN DE PROBLEMAS **6**
1. Diseño 6
2. Estudio teórico 7
3. Programación del algoritmo 8
4. Estudio experimental 9
5. Contraste 10
BACKTRACKING: RESOLUCIÓN DE PROBLEMAS **11**
1. Diseño 11
2. Estudio teórico 12
3. Programación del algoritmo 13
4. Estudio experimental 14
5. Contraste 15
CONCLUSIONES **16**

LISTA DE PROBLEMAS RESUELTOS

Problemas asignados:

- **Avance rápido:** I_AR
- **Backtracking:** L_Ba

Enunciado **I_AR**:

Organización en Clase

Contexto

La asignación de los puestos a los alumnos en una clase es uno de los problemas más complicados a los que se enfrentan los profesores de secundaria, pues hay que tener en cuenta las relaciones personales de los alumnos (la amistad entre ellos) pero también la compenetración en el trabajo. En este problema se trata de agrupar los alumnos de dos en dos, pues suponemos que los pupitres son de dos plazas. No tendremos en cuenta la posición de los pupitres en la clase y supondremos que lo único que se pretende es agrupar a los alumnos de dos en dos (si el número es impar uno quedará sólo en un pupitre) intentando maximizar la amistad y al mismo tiempo la compenetración en el trabajo de los alumnos que sentamos juntos.

El Problema

Escribir un programa que realice una agrupación de alumnos de dos en dos maximizando la suma de los productos del grado de amistad y la compenetración en el trabajo de alumnos que se agrupan juntos. Se dispone para ello de una matriz de amistad y otra de trabajo, donde se guardan los grados de amistad y de compenetración, que pueden no ser recíprocos, por lo que las matrices no son simétricas. La amistad y la compenetración de un alumno consigo mismo se almacenará como cero. En cada pupitre se suman los grados de amistad de los alumnos en el pupitre, y se suman los grados de compenetración, y se multiplican los resultados de las dos sumas. El valor de beneficio es la suma de los productos obtenidos en todos los pupitres (sin incluir el posible pupitre donde sólo haya un alumno).

Suponer, por ejemplo, un caso donde el número de alumnos N = 3. Sean las matrices de amistad y trabajo las siguientes:

Amistad	*Trabajo*
0 5 6	0 5 3
4 0 3	3 0 2
2 1 0	1 5 0

Si numeramos los alumnos con 0, 1 y 2, la agrupación <0, 1> tendrá valor 72 (que sale de: (5+4)*(5+3)); la <0, 2> valor 32; y la <1, 2> valor 28, por lo que la mejor opción es sentar a los alumnos 0 y 1 juntos y dejar al 2 sólo.

Habrá que resolver el problema con avance rápido. La solución no tiene por qué ser la óptima, y se admitirán soluciones que no empeoren en más de un 30% de las obtenidas con el programa de los profesores (en el ejemplo la única admisible será la de la agrupación <0, 1>).

Se considerará que el número máximo de alumnos es 100.

Entrada

La primera línea de la entrada contiene un entero, P, que indica el número de casos de prueba.

Para cada caso de prueba, la primera línea contiene un entero, N, que indica el número de alumnos.

A continuación, aparecen N líneas (una por alumno) en la entrada para la matriz de amistad, seguidas de otras N líneas (una por alumno) para la de trabajo. Cada línea contiene a su vez N-1 enteros que representan los grados de amistad y compenetración del correspondiente alumno con los restantes. Los valores en esas líneas estarán entre 1 y 9, y estarán separados por espacios en blanco. Obsérvese que se omiten los valores 0 de cada alumno consigo mismo.

Salida

La salida constará de 2*P líneas. Las líneas impares contendrán el valor de beneficio total obtenido para cada uno de los casos de prueba. Las líneas pares contendrán una secuencia de N valores, uno por cada alumno de cada caso de prueba. La secuencia indica los emparejamientos realizados: los valores en posiciones impares corresponden al primer alumno de cada pareja, y los de posiciones pares al segundo. Si N es impar, el último alumno queda desparejado.

Ejemplo de Entrada	**Ejemplo de Salida**
3	72
3	0 1 2
5 6	40
4 3	0 1
2 1	51
5 3	3 1 0 2
3 2	
1 5	
2	
3	
2	
1	
7	
4	
1 2 3	
1 2 3	
1 2 3	
1 2 3	
3 2 4	
2 3 4	
5 3 1	
3 2 4	

Enunciado **L_Ba**:

Mi Camiseta me Viene

Contexto

Nuestro amigo Victor participa como un instructor en un programa de voluntariado. El jefe de Victor le ha pedido que distribuya N camisetas entre M voluntarios, una camiseta para cada voluntario, donde N es múltiplo de seis, y N >= M. Hay el mismo número de camisetas de cada una de las 6 tallas disponibles: XXL, XL, L, M , S, y XS. Victor tiene un pequeño problema porque a cada voluntario sólo le vienen dos tallas de camiseta y no sabe si tendrá las tallas adecuadas para todos.

Problema

El problema consiste en escribir un programa que decida si Victor puede distribuir las camisetas de tal modo que todos los voluntarios tengan una camiseta que les venga bien. Téngase en cuenta que si N > M pueden sobrar algunas camisetas.

Entrada

La primera línea de la entrada contiene el número total de casos de prueba.

Para cada caso de prueba, hay un línea con dos números N y M. N es múltiplo de 6, 1 <= N <= 36, e indica el número de camisetas. El número M, 1 <= M <= 36, indica el número de voluntarios, con N >= M. A continuación vienen M líneas donde cada línea contiene dos tallas separadas por un espacio, que indican la dos tallas que le vienen a ese voluntario (XXL, XL, L, M , S o XS).

Salida

Para cada caso de prueba hay que imprimir una línea con el texto YES si hay, al menos, una posible distribución donde cada voluntario tenga una camiseta de su talla, o NO en caso contrario.

Ejemplo de Entrada	**Ejemplo de Salida**
3 18 6 L XL XL L XXL XL S XS M S M L 6 4 S XL L S L XL L XL 6 1 L M	YES NO YES

Problema	Envío	Implementación	Causa fallos
I_AR			
L_Ba			

AVANCE RÁPIDO: RESOLUCIÓN DE PROBLEMAS

1. Diseño

Diseño de una solución por avance rápido; incluyendo pseudocódigo y explicación de los algoritmos, justificando las decisiones de diseño, las estructuras de datos y las funciones básicas del esquema algorítmico.

2. Estudio teórico

Estudio teórico del tiempo de ejecución del algoritmo. Especialmente importante t_M, decir lo que se pueda de t_m si tiene sentido.

3. Programación del algoritmo

Programación del algoritmo (debe ser aceptado por mooshak). El programa debe ir documentado, con explicación de qué es cada variable, qué realiza cada función y su correspondencia con las funciones básicas del esquema algorítmico correspondiente. Adjuntar los ficheros de código en la entrega, pero no incluir el código en la memoria. En la memoria explicar qué ficheros se han utilizado, qué hace cada uno, cómo se usan (al estilo del ejemplo de práctica proporcionado).

4. Estudio experimental

Estudio experimental del tiempo de ejecución para distintos tamaños de problema. Especialmente importante tM, si puede caracterizarse, o en su defecto tp. Indicar qué tamaños de entrada se han probado (usar espaciado multiplicativo) y mostrar una gráfica con los puntos obtenidos.

5. Contraste

Contraste teórico-experimental. Usar regresión, mostrar gráfica con los tiempos experimentales y la regresión, y mostrar y comentar R2.

BACKTRACKING: RESOLUCIÓN DE PROBLEMAS

1. Diseño

2. Estudio teórico

3. Programación del algoritmo

4. Estudio experimental

5. Contraste

CONCLUSIONES

Incluir las conclusiones y valoraciones personales de la actividad, y la estimación del tiempo total que se ha tardado en completarla (por cada alumno/a, en horas).